



Práctica 3. Interfaces como contrato y menús de interacción con el usuario.

Objetivos de aprendizaje

1. Comprender y aplicar el concepto de interfaz en Java como contrato de comportamiento.	Entender cómo una interfaz define un conjunto de métodos que las clases deben implementar, garantizando así un comportamiento común (contrato).
2. Comprender el papel de las interfaces como contrato y su uso en clases de contexto.	Conocer y entender que una interfaz define un contrato de comportamiento común y que, al usar referencias de este tipo en estructuras como <code>JuegosReunidos</code> , se logra no solo homogeneizar el acceso a diferentes clases, sino también facilitar la ampliación del sistema con nuevos juegos que implementen la interfaz.
3. Diseñar e implementar menús de interacción con el usuario	Desarrollar menús que permitan al usuario seleccionar y ejecutar distintas funcionalidades de la aplicación.
4. Convenciones de codificación en Java	Aplicar adecuadamente las normas de estilo y convenciones de codificación de Java (nombres de variables, formato, comentarios, indentación, etc.), mostrando código claro y legible.

Descripción

En esta práctica se introducirá el concepto de interfaz en Java como contrato de comportamiento y se construirá un menú de interacción con el usuario. El uso de la interfaz **Jugable** permitirá no solo acceder de forma uniforme a objetos de distintas clases, sino también ampliar el sistema con nuevos juegos sin modificar el código existente, continuando así la evolución iniciada en las prácticas anteriores. Además, la clase de contexto **JuegosReunidos** y la clase **Menu** reforzarán, en cierta medida, la separación entre la lógica del modelo y la interfaz de usuario, favoreciendo la mantenibilidad y escalabilidad de la aplicación.

1. Primera parte. Interfaces

En esta primera parte del ejercicio se va a implementar una interfaz **Jugable** que implementarán los juegos desarrollados hasta ahora, y los que se desarrollen en un futuro.

1.1. Modificación de la Clase JuegoConVidas

- Se eliminará su método abstracto **juega**, pero la clase se seguirá manteniendo como abstracta ya que no interesa que se creen instancias de esta clase.

1.2. Creación del Interfaz Jugable

- Dispondrá de un método **juega** con las mismas características que el método **juega** que se eliminó de la clase JuegoConVidas.
- Se incorporará el método **reiniciaPartida** con la misma cabecera que tiene el método en la clase JuegoConVidas.
- Se incorporará un método **muestraNombre** que no tome ningún parámetro y que obligue a las clases que implementen la interfaz a mostrar un mensaje por pantalla con el nombre del juego.
- Se incorporará un método **muestraInfo** que no tome ningún parámetro y que obligue a las clases que implementen la interfaz a mostrar un mensaje por pantalla con una descripción de cómo jugar al juego.

1.3. Modificación de la Clase JuegoAdivinaNumero

- Debe implementar la interfaz Jugable.
- Implementar el método **muestraNombre** que visualizará por pantalla el texto "Adivina un número".
- Implementar el método **muestraInfo** que visualizará por pantalla una descripción de cómo se juega al juego, informando del número de intentos que se le dan al jugador.

1.4. Modificación de la Clase JuegoAdivinaNumeroPar

- Reescribir el método **muestraNombre** para que visualice por pantalla el texto "Adivina un número par".
- Reescribir el método **muestraInfo** para que visualice la información de este nuevo juego.

1.5. Modificación de la Clase JuegoAdivinaNumeroImpar

- Reescribir el método **muestraNombre** para que visualice por pantalla el texto Adivina un número impar
- Reescribir el método **muestraInfo** para que visualice la información de este nuevo juego.

1.6. Modificar la clase principal

- Modificación del método **jugar**: Este método estático, no devuelve ningún valor y recibe como parámetro un objeto que implementa la interfaz Jugable. Su objetivo es gestionar el desarrollo completo de una partida. En primer lugar, muestra el nombre y la información del juego mediante las llamadas a **muestraNombre()** y **muestraInfo()**, y reinicia la partida con **reiniciaPartida()**. A continuación, declara una variable booleana **flag** que actúa como bandera de control del bucle. Dentro del bucle **do-while**, se solicita al usuario que introduzca su respuesta mediante **MyInput.readString()** y se ejecuta la jugada llamando a **j.juega(car)**, asignando su resultado a la variable **flag**. Mientras **flag** sea true, el bucle continúa y el juego sigue en marcha; cuando **flag** pasa a false, la partida finaliza y el método termina su ejecución.
- Por último, el método **main** creará de nuevo una instancia de cada uno de los tres juegos implementados, pero el tipo de referencia que señalará a

dichos objetos será de tipo **Jugable**. Como en el ejercicio anterior, el número de vidas de cada juego será 5 y como número a adivinar un número cualquiera, otro par y otro impar respectivamente, todos comprendidos entre 0 y 10.

2. Segunda parte. Menú de usuario

En esta segunda parte se introduce el concepto de menú de usuario como la clase, o colección de clases, donde el sistema interacciona con el usuario a la hora de intercambiar información.

La clase **Menu** hará uso de una clase, denominada **JuegosReunidos**, donde se centralizarán todos los recursos del programa (los distintos juegos). En esta clase de contexto se hará uso de un array de objetos de tipo **Jugable**, lo que permitirá acceder a cualquiera de los tres juegos a través de dicha estructura de datos.

2.1. Crear la clase **JuegosReunidos**

La clase **JuegosReunidos** actúa como contenedor central de todos los juegos disponibles en la aplicación y permite gestionarlos de forma sencilla (clase de contexto). Su función principal es almacenar los objetos de tipo **Jugable** que podrán ser seleccionados y ejecutados por el usuario. Como atributo, dispone de un array de tipo **Jugable** en el que se guardará una instancia de cada juego. Este array se inicializa en el constructor de tal forma que crea un contenedor con tres posiciones inicialmente vacías (null). Dispone además de tres métodos: **agregar**, **recuperarJuego** y **numeroJuegos**.

El método **agregar(Jugable j, int i)** permite añadir un juego concreto al array. Para implementarlo, se recibe como parámetros el objeto **j**, que representa el juego a guardar, y el índice **i**, que indica la posición del array donde se almacenará. Dentro del método, se lleva a cabo la asignación.

El método **recuperarJuego(int opcion)** devuelve el juego que el usuario ha elegido. Como el menú de opciones se muestra numerado a partir de 1, pero los arrays en Java comienzan en 0, es necesario ajustar la posición restando una unidad. De este modo, si el usuario introduce "1", se accederá al primer elemento del array.

Finalmente, el método **numeroJuegos()** recorre el array contando cuántos juegos han sido efectivamente cargados. Para ello, se inicializa un contador en 0 y se recorre el array, incrementando el contador cada vez que se encuentra una posición no nula. Al terminar, se devuelve el valor del contador, que representa el número total de juegos disponibles para mostrar en el menú.

2.2. Crear la clase **Menu**

La clase **Menu** representa la entidad que se encarga de gestionar la interacción con el usuario. Esta clase contendrá un atributo de la clase **JuegosReunidos** de tal forma que le permita acceder a los distintos recursos que centraliza. El constructor de esta clase deberá tener como parámetro una referencia de tipo **JuegosReunidos** que utilizará para inicializar su atributo.

Además del constructor, esta clase tendrá los siguientes métodos:

- Método **jugar**: es el mismo método que se implementó en la clase principal en el apartado anterior, pero en este caso el método es privado y no estático.

- Método **mostrarOpciones**: Es un método privado que muestra por consola el siguiente mensaje:
 ¿A qué quieres jugar?
 Introduce un 1 si quieres jugar a Adivinar un número.
 Introduce un 2 si quieres jugar a Adivinar un número par.
 Introduce un 3 si quieres jugar a Adivinar un número i
 El método imprime primero el mensaje “¿A qué quieres jugar?”, y a continuación recorre los juegos disponibles mediante un bucle **for** que va desde 1 hasta el número total de juegos cargados. En cada iteración muestra el texto “Introduce un (número de iteración) si quieres jugar a”, seguido del nombre del juego, que se obtiene invocando el método **muestraNombre()** sobre el juego recuperado con **recuperarJuego(...)**. De este modo, el usuario puede visualizar todas las opciones y seleccionar el juego deseado introduciendo el número correspondiente.
- Método **chequeoOpcion**: Es un método privado que tiene como parámetro una variable de tipo entero (la respuesta del usuario) y que retorna un booleano. Este método comprueba si el número que recibe se corresponde a una opción válida de juego. Para ello, verifica si el valor está fuera del rango permitido (menor o igual que 0 o mayor que el número de Juegos cargados). Si la opción no es válida, muestra un mensaje de error por consola y devuelve true para indicar que hubo un error; en caso contrario, devuelve false y el programa puede continuar normalmente..
- Método **eligeOpcion**: Es un método privado que permite al usuario seleccionar una de las opciones disponibles, validando que la elección sea correcta. No recibe parámetros y devuelve un número entero. Su funcionamiento se basa en un bucle que, en cada iteración, muestra las opciones disponibles mediante **mostrarOpciones()**, solicita al usuario que introduzca su elección con **MyInput.readInt()** y llama al método **chequeoOpcion(...)** para comprobar si el número es válido. Si la opción introducida es correcta, el bucle finaliza y el método devuelve dicho valor; en caso contrario, se muestra un mensaje de error por consola y se repite el proceso hasta que el usuario introduzca una opción válida.
- Método **ejecuta**: Es un método público que no tiene parámetros de entrada ni valor de retorno. Su función es ejecutar el juego seleccionado y ofrecer al usuario la posibilidad de volver a jugar una vez finalizada la partida. Para ello, dentro de un bucle, se invoca el método **jugar**, pasándole como parámetro el resultado de **juegos.recuperarJuego(eligeOpcion(...))**, que determina el juego elegido por el usuario. Al finalizar la ejecución del juego, se pregunta al usuario si desea seguir jugando; si introduce la letra “s”, el bucle se repite y se vuelve a mostrar el menú de opciones. Cualquier otro carácter provocará la salida del bucle y la finalización del método.

2.3. Modificación de la clase principal

En la clase Aplicación, el método principal debe encargarse de preparar y lanzar el sistema de juegos. Primero se crea un objeto de tipo **JuegosReunidos**, que actuará como contenedor de todos los juegos disponibles. A continuación, se crean tres instancias concretas de los juegos (**JuegoAdivinaNumero**,

JuegoAdivinaNumeroPar y **JuegoAdivinaNumeroImpar**), cada una configurada con 5 vidas, y se añaden al objeto **JuegosReunidos** mediante el método **agregar**. Una vez almacenados los juegos, se crea un objeto de tipo **Menu**, pasándole por parámetro la referencia al objeto **JuegosReunidos** previamente creado. Por último, se invoca el método **ejecuta()** del objeto **Menu**, con lo que se inicia la interacción con el usuario y la ejecución de los juegos.

3. Evaluación y retroalimentación

3.1. Rúbrica de logro de resultados de aprendizaje

Criterio	Consigue el resultado	No consigue el resultado
1. Comprensión y uso de interfaces en Java	Define e implementa correctamente una interfaz, y explica su papel como contrato de comportamiento y las garantías que ofrece a nivel de diseño y mantenimiento.	No define o implementa correctamente la interfaz, o no comprende su función y utilidad en el diseño de programas Java.
2. Uso de interfaces para la abstracción y extensibilidad del diseño	Utiliza correctamente referencias de tipo interfaz (por ejemplo, Jugable) para acceder de forma uniforme a distintas clases, mostrando cómo este contrato facilita la incorporación de nuevas implementaciones y aporta extensibilidad y mantenimiento al diseño (ej. JuegosReunidos).	No aplica correctamente el uso de referencias de tipo interfaz o las limita a una homogeneización superficial, sin comprender el papel de las interfaces como contrato ni su utilidad en el diseño de sistemas extensibles y desacoplados.
3. Diseño e implementación de menús de usuario	Desarrolla menús funcionales que permiten seleccionar y ejecutar distintas funcionalidades, gestionando correctamente la interacción y la validación de opciones del usuario.	Los menús no funcionan correctamente o no gestionan adecuadamente la entrada y validación de opciones del usuario.
4. Convenciones de codificación en Java	Aplica correctamente las normas de estilo y convenciones de codificación en Java (nombres, formato, comentarios, indentación), mostrando código claro y legible.	No aplica correctamente las normas de estilo Java o no demuestra comprender su importancia para la legibilidad y mantenimiento del código.

3.2. Preguntas de autoevaluación

Responde a las siguientes preguntas, justificando tus respuestas con fragmentos de código o ejemplos concretos desarrollados en la práctica:

1. ¿Qué es una interfaz en Java y qué significa que actúe como un “contrato”?
Explica con tus palabras qué implica que una clase implemente la interfaz Jugable y qué garantías ofrece esto al programador.
2. ¿De qué manera el uso del interfaz Jugable garantiza un acceso uniforme a juegos de distintas clases y, al mismo tiempo, posibilita la incorporación de nuevos juegos sin modificar el código existente?. ¿Qué ventajas aporta el uso de un tipo abstracto (Jugable) frente a trabajar directamente con clases concretas como JuegoAdivinaNumero o JuegoAdivinaNumeroPar?.
3. ¿Qué ocurre si una clase no implementa todos los métodos definidos en la interfaz? Explica las consecuencias que tendría en tu programa.
4. Describe el funcionamiento del menú de usuario implementado. ¿Cómo se gestionan las opciones y la validación de la entrada del usuario?. Explica los métodos principales.
5. ¿Qué ventajas aporta la centralización de los juegos en la clase JuegosReunidos respecto a gestionarlos por separado?
6. ¿De qué manera te ayuda la arquitectura basada en interfaces y menús a mantener o ampliar tu aplicación en el futuro?. Explica cómo podrías añadir un nuevo juego y qué cambios serían necesarios en el código existente.

3.3. Obteniendo retroalimentación de la IAG

A continuación, deberás solicitar a la IAG con la que trabajes una primera evaluación (retroalimentación) sobre la corrección de tu código y las respuestas que has dado a las preguntas planteadas. Para ello elabora un prompt lo más detallado posible que permita a la IAG realizar una evaluación de la corrección tanto del código como de las respuestas proporcionadas. Es importante que en tu prompt incluyas:

- El enunciado de la práctica
- Los objetivos de aprendizaje
- La rúbrica de logro de resultados de aprendizaje.
- El código desarrollado
- Las preguntas de autoevaluación junto con tus respuestas

La valoración recibida **no debe ser una nota numérica**, sino un análisis en el que se indiquen:

- Qué aspectos están bien resueltos.
- Qué errores o carencias se detectan.
- Qué aspectos son mejorables.
- Si se han alcanzado los objetivos y resultados de aprendizaje previstos, tanto en el código como en las respuestas teóricas.

Es fundamental que tu prompt recoja con claridad todos los aspectos mencionados, de modo que la IAG oriente su análisis exactamente a lo que se espera en la evaluación. Si la respuesta de la IAG no es suficientemente clara, ajustada o útil,

mejora tu prompt añadiendo explicaciones adicionales, ejemplos concretos o aclaraciones sobre lo que necesitas.

Recuerda: **la calidad de la retroalimentación recibida depende de la precisión y el contexto que incluyas en tu consulta.** Compara siempre la respuesta de la IAG con los materiales de clase u otras fuentes fiables, o consulta con el/la profesor/a en caso de dudas persistentes. Esto te ayudará a desarrollar un uso crítico y responsable de la IAG.

4. Informe que debe subirse al campus virtual de la asignatura

Una vez obtengas una respuesta satisfactoria, sube a la tarea correspondiente en Moodle un informe en PDF, **utilizando para ello la plantilla que está disponible en el campus virtual.** Este informe incluye:

- Nombre y apellidos
- La IAG utilizada, su versión (si procede) y la fecha.
- El desarrollo de la práctica: El código de la práctica y las respuestas a las preguntas que se formulan en esta práctica.
- Interacción con la IAG: Transcribe el diálogo íntegro y ordenado (tus prompts y las respuestas de la IAG).
- Una valoración personal sobre la utilidad y limitaciones de la retroalimentación recibida de la IAG, indicando qué mejoras podrás incorporar gracias a ella (si las hubiera) y reflexionando brevemente sobre su impacto en tu aprendizaje.
- La declaración de autoría y uso de IAG, conforme a lo indicado en el siguiente apartado.

Puedes aprovechar las observaciones de la IAG para mejorar el código o tus respuestas en la práctica siguiente.

Nota: Es obligatorio entregar el informe en PDF utilizando la plantilla que se facilita en el campus virtual de la asignatura y completar correctamente todos sus apartados; cualquier entrega que no se ajuste a esta plantilla —por ejemplo, subir solo el código o un PDF que no se corresponda con la plantilla— se considerará no válida y conllevará el suspenso de toda la práctica guiada (las cinco entregas).

5. Uso de la IAG, declaración de autoría y buenas prácticas

El uso de herramientas de IAG está permitido en esta práctica únicamente para obtener retroalimentación y sugerencias de mejora sobre el trabajo realizado (por ejemplo, evaluación de código, propuestas de corrección o ayuda en la redacción final). No se permite la elaboración inicial de código, respuestas o informes previos mediante IAG. Todo el desarrollo y resolución de la práctica debe ser realizado personalmente por el estudiante.

Para asegurar la transparencia, la responsabilidad y el uso adecuado de la IAG, la plantilla para la elaboración del informe incluye un apartado final de “Declaración de autoría y uso de IAG”. Mediante esta declaración, el estudiante manifiesta que el trabajo presentado ha sido realizado por él/ella, que asume la responsabilidad sobre su contenido y sobre las decisiones adoptadas durante su elaboración, y que la información proporcionada en el informe acerca del uso de herramientas de Inteligencia Artificial Generativa es completa y veraz.

La entrega del informe supone la aceptación expresa de esta declaración y la asunción de responsabilidad sobre el trabajo presentado.

Recuerda: El uso responsable de la IAG contribuye al aprendizaje autónomo y a la mejora continua, pero no puede sustituir al desarrollo personal de competencias técnicas y críticas. El falseamiento de la autoría será considerado una mala praxis académica.